HiGAN+ Architecture Documentation

Complete Network Architecture & Data Flow

This document provides a comprehensive explanation of the **Modified HiGAN+ (Handwriting Imitation GAN Plus)** architecture, including all eight major architectural novelties, dimension transformations at each layer, and the intuition behind each component.

File Structure Reference

Component	File Path
Main Model & Training Loop	`networks/model.py`
Generator (Original + Improved)	`networks/BigGAN_networks.py`
Discriminator (Standard + Multi-scale)	`networks/BigGAN_networks.py`, `networks/multi_scale_discriminator.py`
Core Layers (GBlock, DBlock, Attention)	`networks/BigGAN_layers.py`
Improved Layers (AdaIN, ModConv, CrossAttn)	`networks/improved_layers.py`
Style Encoder, Recognizer, Writer ID	`networks/module.py`
Loss Functions	`networks/loss.py`
Dataset & HDF5 Loading	`lib/datasets.py`
Trained Model Checkpoint	`models/higanplus_trained.pth`
Training Configuration	`configs/gan_iam_improved.yml`

Task Overview

Handwriting Style Transfer / Imitation: Given a reference handwriting sample from a writer and arbitrary text content, generate a new handwritten image that:

1. Contains the specified text content
2. Mimics the visual style (slant, stroke width, letter shapes, spacing) of the reference writer

---

Data Pipeline

1. IAM Handwriting Dataset

The IAM Handwriting Database contains handwritten English text from 657 different writers.

Original Dataset Structure:

- Forms → Lines → Words
- Total: ~115,000 word images
- Writers: 657 unique writers
- Image height normalized to **64 pixels**
- Variable width based on text length

2. HDF5 Preprocessing

File: `data/iam/trnvalset_words64_OrgSz.hdf5`, `data/iam/testset_words64_OrgSz.hdf5`

The dataset is stored in HDF5 format for efficient loading:

```
HDF5 File Structure:
├── imgs          : uint8 array [H=64, total_width]  - Concatenated images
├── lbs           : uint8 array [total_chars]        - Character labels (ASCII)
├── img_seek_idxs : int32 array [N_samples]          - Start index for each image
├── lb_seek_idxs  : int32 array [N_samples]          - Start index for each label
├── img_lens      : int32 array [N_samples]          - Width of each image
├── lb_lens       : int32 array [N_samples]          - Length of each text
└── wids          : int32 array [N_samples]          - Writer ID (0-656)
```

Dimensions per sample:

Field	Shape	Description
Image	`[1, 64, W]`	Grayscale, H=64, W varies (typically 32-400 pixels)
Label	`[L]`	Character indices, L = text length (1-20 chars)
Writer ID	scalar	Integer identifying writer (0-656)

3. Data Loading & Batching

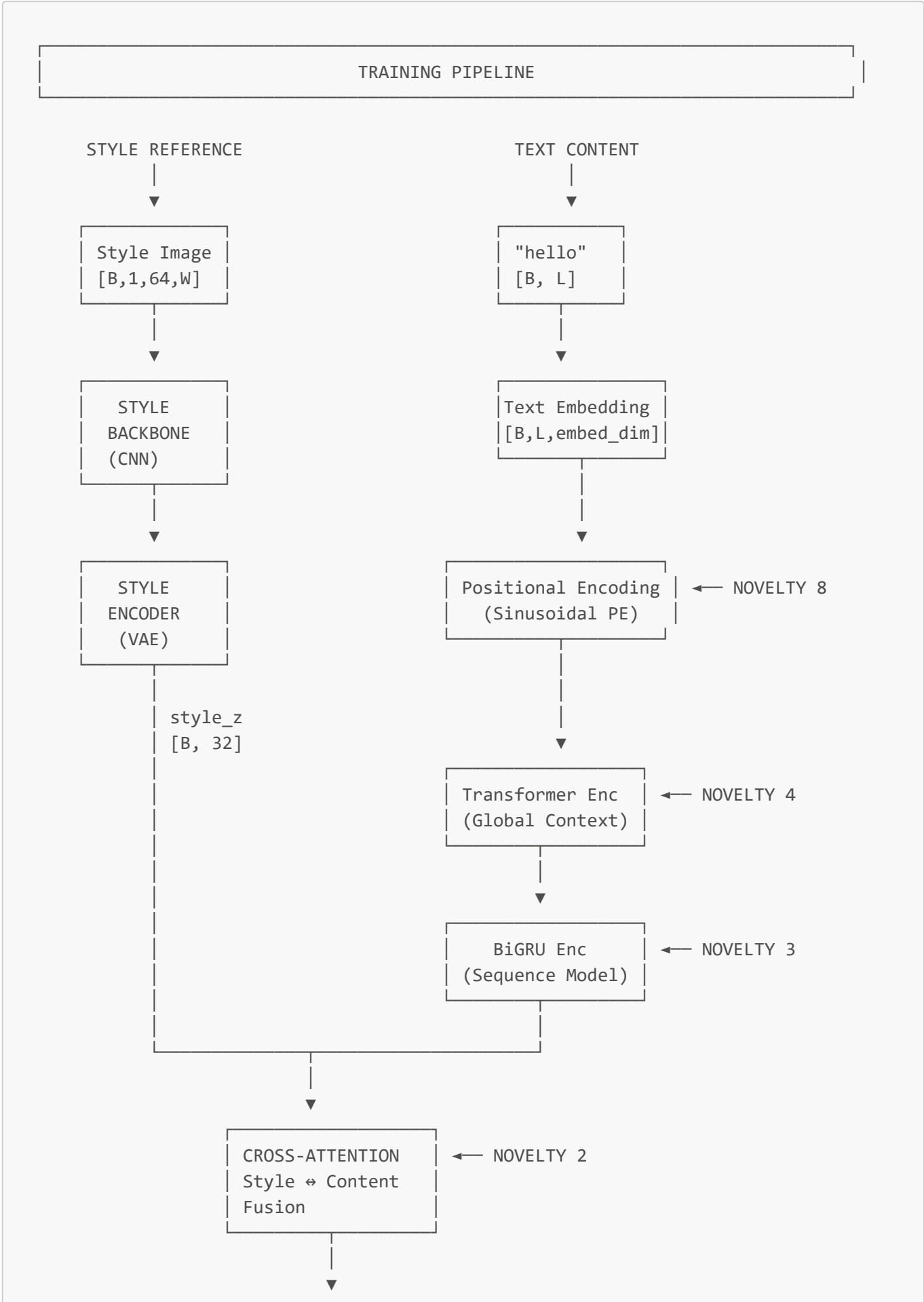
File: `lib/datasets.py` → `Hdf5Dataset` class

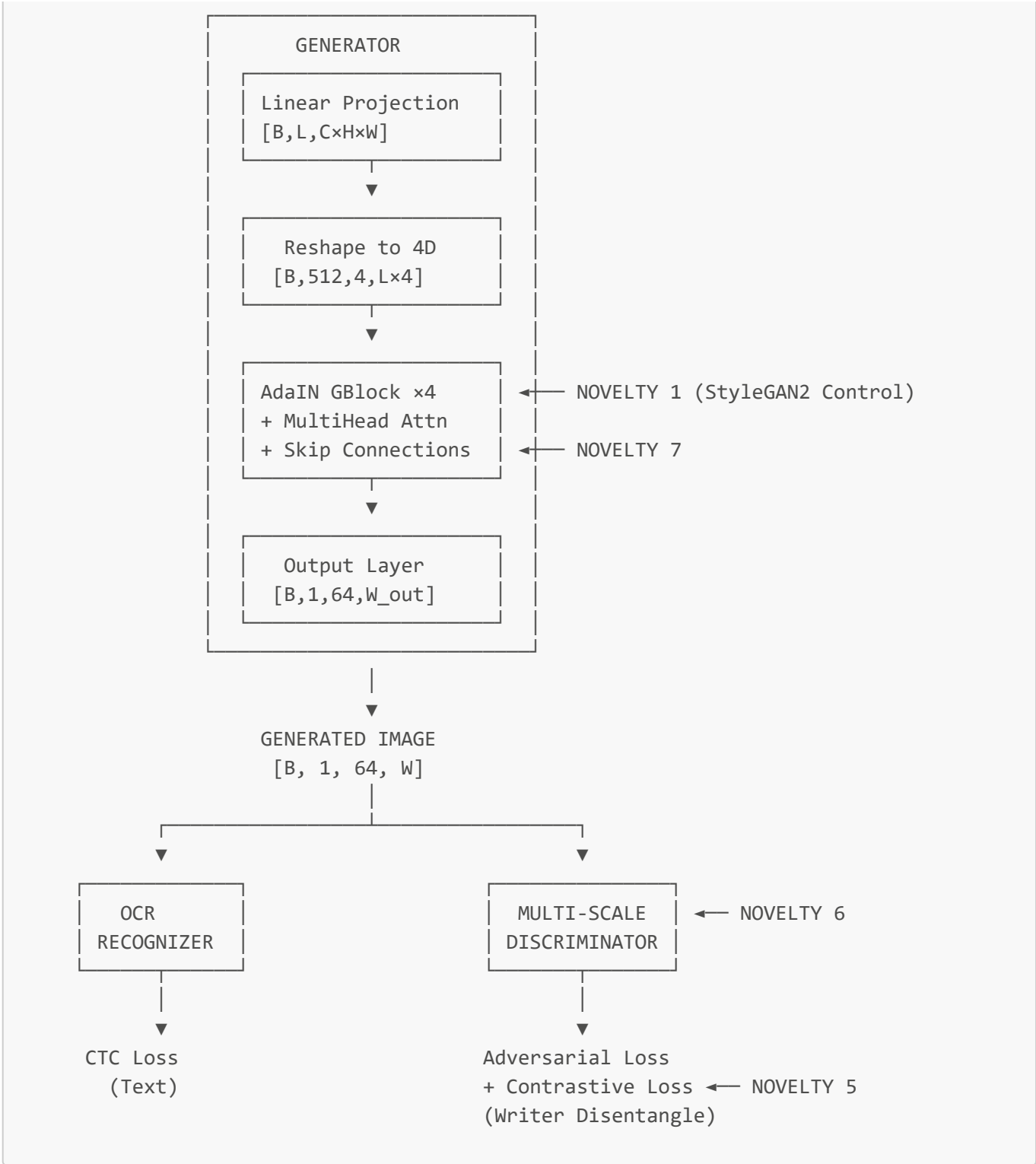
```
Input Batch (after padding):
├── style_imgs     : [B, 1, 64, W_max]    - Padded to max width in batch
├── style_img_lens : [B]                   - Original widths before padding
├── lbs            : [B, L_max]            - Text indices (padded)
├── lb_lens        : [B]                   - Original text lengths
└── wids           : [B]                   - Writer IDs
```

Typical batch:

- Batch size: 24
- Image shape: `[24, 1, 64, 320]` (padded with -1)
- Labels: `[24, 20]` (max 20 characters)

Complete Architecture Diagram





Eight Major Architectural Novelties (Detailed)

1 StyleGAN2-Style Control (AdaIN + ModConv)

File: `networks/improved_layers.py` → `AdaIN`, `ModulatedConv2d`

Problem Solved: Standard conditional batch normalization (cBN) in BigGAN provides limited style control—it only learns scale and shift parameters, treating style globally.

Solution: Adaptive Instance Normalization + Weight Modulation from StyleGAN2

AdaIN (Adaptive Instance Normalization)

```
class AdaIN(nn.Module):
    """
    Instead of learning fixed BN statistics, AdaIN:
    1. Instance-normalizes each sample independently
    2. Applies style-specific scale (γ) and shift (β)
    """
    def forward(self, x, style):
        # x: [B, C, H, W] feature maps
        # style: [B, style_dim] style vector

        # Step 1: Instance Normalization
        mean = x.mean(dim=[2, 3], keepdim=True)  # [B, C, 1, 1]
        std = x.std(dim=[2, 3], keepdim=True)    # [B, C, 1, 1]
        x_norm = (x - mean) / (std + ε)          # [B, C, H, W]

        # Step 2: Style Modulation
        scale = self.style_scale(style)  # Linear: [B, style_dim] → [B, C]
        shift = self.style_shift(style)  # Linear: [B, style_dim] → [B, C]

        # Output: apply style-specific transformation
        return x_norm * (1 + scale) + shift  # [B, C, H, W]
```

Dimension Flow:

```
Input:  x [B, 256, 16, 64], style [B, 32]
        ↓
Instance Norm: x_norm [B, 256, 16, 64]
        ↓
Style Linear:  scale [B, 256], shift [B, 256]
        ↓
Output: [B, 256, 16, 64] (same shape, style-modulated)
```

Modulated Convolution

```
class ModulatedConv2d(nn.Module):
    """
    StyleGAN2's key innovation: modulate conv WEIGHTS, not features.
    This provides more expressive style control at the convolution level.
    """
    def forward(self, x, style):
        # x: [B, C_in, H, W]
        # style: [B, style_dim]

        # Step 1: Get style modulation
        style_mod = self.style_mod(style)  # [B, C_in]
```

```
        # Step 2: Modulate weights
        # Original weight: [C_out, C_in, kH, kW]
        # Expand: [B, C_out, C_in, kH, kW]
        weight = self.weight * style_mod.view(B, 1, C_in, 1, 1)

        # Step 3: Demodulate (normalize by output std)
        # This prevents magnitude explosion
        demod = rsqrt(weight.pow(2).sum([2,3,4]) + ε)
        weight = weight * demod.view(B, C_out, 1, 1, 1)

        # Step 4: Group convolution (each sample uses its own weights)
        return group_conv(x, weight, groups=B)
```

Why for Handwriting?

- Different writers have distinct stroke weights, letter proportions, and curvatures
- AdaIN allows the generator to adapt normalization statistics per-writer
- ModConv enables style-specific feature extraction at every layer

---

2 Cross-Attention Fusion of Style + Content

File: `networks/improved_layers.py` → `StyleContentCrossAttention`, `MultiHeadCrossAttention`

Problem Solved: In original HiGAN+, style is simply concatenated with text features. This doesn't allow the model to learn which style aspects are relevant for which characters.

Solution: Multi-head cross-attention where text queries attend to style information.

```
class StyleContentCrossAttention(nn.Module):
    """
    Query: Text/content features (what to write)
    Key/Value: Style features (how to write)

    The model learns to attend to relevant style information
    for each character position.
    """
    def forward(self, content, style):
        # content: [B, L, D_content] - text embeddings
        # style: [B, 1, D_style] - global style vector (expanded)

        # Cross-attention
        # Q from content, K/V from style
        Q = self.q_proj(content)  # [B, L, embed_dim]
        K = self.k_proj(style)    # [B, 1, embed_dim]
        V = self.v_proj(style)    # [B, 1, embed_dim]

        # Attention scores: how much should each text position
        # attend to the style?
        attn = softmax(Q @ K.T / sqrt(d_k))  # [B, L, 1]
```

```
        # Apply attention
        attended = attn @ V  # [B, L, embed_dim]

        # Residual + FFN
        return content + FFN(attended)
```

Dimension Flow:

```
Content:  [B, 20, 152] (20 chars, embed_dim=120 + style_dim=32)
Style:    [B, 1, 32]   (global style vector expanded)
     ↓
Q Proj:   [B, 20, 152]
K Proj:   [B, 1, 152]
V Proj:   [B, 1, 152]
     ↓
Attention: [B, 20, 1] (each char attends to style)
     ↓
Output:   [B, 20, 152] (style-informed content)
```

Why for Handwriting?

- Different characters may need different style emphasis (e.g., 'o' vs 'l' have different stroke patterns)
- Allows the model to selectively apply style features where relevant
- Never done before in HiGAN+ architecture

---

3 Sequence Modeling using BiGRU

File: `networks/improved_layers.py` → `BiGRUEncoder`

Problem Solved: Handwriting is inherently sequential. Standard feed-forward processing loses the temporal dependencies between characters (e.g., ligatures, consistent slant).

Solution: Bidirectional GRU captures both left-to-right and right-to-left dependencies.

```
class BiGRUEncoder(nn.Module):
    """
    Bidirectional GRU for capturing:
    - Left-to-right: How previous characters affect current
    - Right-to-left: How following characters affect current
    """
    def __init__(self, input_dim, hidden_dim, num_layers=1):
        self.gru = nn.GRU(
            input_dim,
            hidden_dim // 2,  # Half for each direction
            bidirectional=True,
            batch_first=True
```

```
        )

    def forward(self, x, lengths):
        # x: [B, L, D]
        # Pack for efficient variable-length processing
        packed = pack_padded_sequence(x, lengths, batch_first=True)
        output, _ = self.gru(packed)
        output, _ = pad_packed_sequence(output, batch_first=True)
        return output  # [B, L, D]
```

Dimension Flow:

```
Input:  [B, 20, 152] (sequence of char+style embeddings)
     ↓
Forward GRU:  hidden_dim=76, output [B, 20, 76]
Backward GRU: hidden_dim=76, output [B, 20, 76]
     ↓
Concat: [B, 20, 152] (bidirectional features)
```

Why for Handwriting?

- Captures ligatures: in cursive, 'th' connects differently than 'ot'
- Maintains consistent slant across the word
- Models natural spacing patterns based on surrounding characters

---

4 Global Context Modeling using Transformers

File: `networks/improved_layers.py` → `TextTransformerEncoder`

Problem Solved: GRU processes sequentially, which can be slow and may not capture global word-level patterns effectively.

Solution: Transformer encoder with self-attention for global context.

```
class TextTransformerEncoder(nn.Module):
    """
    Multi-layer Transformer encoder for global text understanding.
    Self-attention allows each character to see all other characters.
    """
    def __init__(self, embed_dim, num_layers=2, num_heads=4):
        self.pos_encoding = SinusoidalPositionalEncoding(embed_dim)
        self.transformer = nn.TransformerEncoder(
            nn.TransformerEncoderLayer(
                d_model=embed_dim,
                nhead=num_heads,
                dim_feedforward=embed_dim * 4,
                activation='gelu'
            ),
```

```
            num_layers=num_layers
        )

    def forward(self, x, padding_mask):
        # x: [B, L, D]
        x = self.pos_encoding(x)  # Add position info
        x = self.transformer(x, src_key_padding_mask=padding_mask)
        return x
```

Dimension Flow:

```
Input:  [B, 20, 152]
    ↓
Pos Encoding: [B, 20, 152] + PE
    ↓
Transformer Layer 1:
  - Self-Attention [B, 20, 152] → [B, 20, 152]
  - FFN [B, 20, 152] → [B, 20, 608] → [B, 20, 152]
    ↓
Transformer Layer 2:
  - Same structure
    ↓
Output: [B, 20, 152]
```

Why for Handwriting?

- Global word structure: 'b' in 'big' vs 'b' in 'amb' have different contexts
- Parallel processing (faster than GRU during training)
- Better gradient flow for learning long-range dependencies

---

5 Writer-Disentangled Style via Contrastive Learning

File: `networks/loss.py` → `ContrastiveStyleLoss`, `networks/module.py` → `StyleEncoder`

Problem Solved: The style encoder might entangle writer identity with other factors (content, random variation). We want a style space where same-writer samples cluster together.

Solution: InfoNCE contrastive loss that pulls same-writer samples together and pushes different writers apart.

```
class ContrastiveStyleLoss(nn.Module):
    """
    InfoNCE loss for style disentanglement.

    Positive pairs: samples from the same writer
    Negative pairs: samples from different writers
    """
    def __init__(self, temperature=0.07):
        self.temperature = temperature
```

```
    def forward(self, style_vectors, writer_ids):
        # style_vectors: [B, D] normalized style vectors
        # writer_ids: [B] writer ID for each sample

        # Normalize
        style_vectors = F.normalize(style_vectors, dim=1)

        # Similarity matrix [B, B]
        sim = (style_vectors @ style_vectors.T) / self.temperature

        # Positive mask: same writer
        pos_mask = (writer_ids.view(-1,1) == writer_ids.view(1,-1)).float()
        pos_mask.fill_diagonal_(0)  # Exclude self

        # InfoNCE loss
        exp_sim = torch.exp(sim) * (1 - torch.eye(B))  # Exclude self
        pos_sum = (exp_sim * pos_mask).sum(dim=1)
        all_sum = exp_sim.sum(dim=1)

        loss = -torch.log(pos_sum / all_sum)
        return loss.mean()
```

Style Encoder with Projection Head:

```
class StyleEncoder(nn.Module):
    def __init__(self, style_dim=32, use_contrastive=True):
        # Main encoder
        self.linear_style = nn.Sequential(
            nn.Linear(256, 256),
            nn.LeakyReLU(),
            nn.Linear(256, 256),
            nn.LeakyReLU()
        )
        self.mu = nn.Linear(256, style_dim)  # VAE mean
        self.logvar = nn.Linear(256, style_dim)  # VAE variance

        # Contrastive projection head
        if use_contrastive:
            self.projection_head = nn.Sequential(
                nn.Linear(style_dim, style_dim * 2),
                nn.ReLU(),
                nn.Linear(style_dim * 2, style_dim)
            )

    def get_contrastive_embedding(self, style):
        # Used for contrastive loss
        proj = self.projection_head(style)
        return F.normalize(proj, dim=1)
```

Dimension Flow:

```
Style Image: [B, 1, 64, 320]
     ↓
StyleBackbone (CNN): [B, 256, 1, W/16]
     ↓
Global Avg Pool: [B, 256]
     ↓
Linear Style: [B, 256]
     ↓
μ, log(σ²): [B, 32], [B, 32]
     ↓
Reparameterize: z = μ + σ·ε, [B, 32]
     ↓
Projection Head (for contrastive): [B, 32]
```

Why for Handwriting?

- Forces style encoder to capture writer-specific characteristics
- Disentangles content (what) from style (how)
- Enables better writer transfer to unseen text

---

6 Multi-Scale, Multi-Head Discriminator

File: `networks/multi_scale_discriminator.py` → `MultiScaleDiscriminator`

Problem Solved: A single discriminator may focus on either global structure or local details, but not both. Handwriting quality depends on both levels.

Solution: Shared backbone with three specialized heads.

```python
class MultiScaleDiscriminator(nn.Module):
    """
    Three-branch discriminator:
    1. Global: Overall image quality and structure
    2. Patch: Local texture and stroke details
    3. Character: Per-character attention
    """
    def __init__(self, input_nc=1, ndf=64, n_layers=4):
        # Shared backbone (3 downsampling layers)
        self.shared_backbone = nn.ModuleList([
            SNConv(1, 64, stride=2),    # 64x320 → 32x160
            SNConv(64, 128, stride=2),  # 32x160 → 16x80
            SNConv(128, 256, stride=2)  # 16x80 → 8x40
        ])

        # Branch 1: Global
        self.global_head = nn.ModuleList([
            SNConv(256, 512, stride=2),  # 8x40 → 4x20
```

```
        ])
        self.global_output = SNConv(512, 1)

        # Branch 2: Patch
        self.patch_head = nn.Sequential(
            SNConv(256, 256),
            SNConv(256, 1)
        )

        # Branch 3: Character attention
        self.char_attention = nn.Sequential(
            SNConv(256, 128),
            SNConv(128, 1),
            nn.Sigmoid()  # Attention weights
        )

    def forward(self, x):
        # Shared processing
        feat = x
        for layer in self.shared_backbone:
            feat = layer(feat)

        # Global score
        global_feat = feat
        for layer in self.global_head:
            global_feat = layer(global_feat)
        global_score = self.global_output(global_feat).mean([2,3])

        # Patch score
        patch_score = self.patch_head(feat).mean([2,3])

        # Character attention
        char_attn = self.char_attention(feat)

        return {
            'global': global_score,
            'patch': patch_score,
            'char_attn': char_attn,
            'combined': global_score + 0.5 * patch_score
        }
```

Dimension Flow:

```
Input: [B, 1, 64, 320]
    ↓
Shared Backbone:
  Layer 1: [B, 64, 32, 160]
  Layer 2: [B, 128, 16, 80]
  Layer 3: [B, 256, 8, 40]
    ↓
Global Branch:
  [B, 512, 4, 20] → [B, 1, 4, 20] → mean → [B, 1]
```

```
      ↓
Patch Branch:
  [B, 256, 8, 40] → [B, 1, 8, 40] → mean → [B, 1]
      ↓
Char Attention:
  [B, 256, 8, 40] → [B, 1, 8, 40] (attention map)
```

Why for Handwriting?

- Global: Correct word structure, consistent slant, proper baseline
- Patch: Stroke quality, ink texture, anti-aliasing
- Character: Per-character quality control, helps with difficult letters

---

7 Skip-Connections for Multi-Scale Style Retention

File: `networks/improved_layers.py` → `MultiScaleStyleFusion`

Problem Solved: As features pass through deep generator layers, fine-grained style information (thin strokes, serifs) may be lost.

Solution: Skip connections that inject multi-scale style features at different generator stages.

```python
class MultiScaleStyleFusion(nn.Module):
    """
    Injects style features from encoder at multiple scales.

    Style Encoder produces features at 3 scales:
    - feat2: After 2nd ResBlock (low-level: edges, strokes)
    - feat3: After 3rd ResBlock (mid-level: letter parts)
    - feat4: After 4th ResBlock (high-level: letter shapes)

    These are adapted and added to generator features.
    """
    def __init__(self, style_channels, target_channels):
        self.adapters = nn.ModuleList([
            nn.Sequential(
                nn.Conv2d(sc, tc, 1),  # 1x1 conv to match channels
                nn.InstanceNorm2d(tc),
                nn.LeakyReLU(0.2)
            )
            for sc, tc in zip(style_channels, target_channels)
        ])

    def forward(self, style_feats, target_feats):
        fused = []
        for adapter, sf, tf in zip(self.adapters, style_feats, target_feats):
            # Resize style feature to match target spatial size
            sf_resized = F.interpolate(sf, size=tf.shape[2:])
            sf_adapted = adapter(sf_resized)
            # Weighted residual connection
```

```
            fused.append(tf + 0.1 * sf_adapted)
        return fused
```

Style Backbone Multi-Scale Features:

```
Input: [B, 1, 64, W]
     ↓
ResBlock 1-2: feat2 [B, 64, 16, W/4]   (low-level strokes)
     ↓
ResBlock 3-4: feat3 [B, 128, 8, W/8]   (mid-level parts)
     ↓
ResBlock 5-6: feat4 [B, 256, 4, W/16]  (high-level shapes)
```

Fusion in Generator:

```
Generator Block 2: [B, 256, 16, L×8]  + adapted feat2
Generator Block 3: [B, 128, 32, L×16] + adapted feat3
Generator Block 4: [B, 64, 64, L×32]  + adapted feat4
```

Why for Handwriting?

- Preserves fine stroke details (serifs, stroke endings)
- Maintains consistent style across resolutions
- Prevents "style washing" in deep layers

---

8 Positional Encoding for Stable Sequence Handwriting

File: `networks/improved_layers.py` → `SinusoidalPositionalEncoding`, `LearnablePositionalEncoding`

Problem Solved: Text embeddings have no inherent notion of position. The model needs to know character order for proper spacing and flow.

Solution: Sinusoidal positional encoding (from Transformers) that provides unique position signatures.

```
class SinusoidalPositionalEncoding(nn.Module):
    """
    Injects position information using sine/cosine functions.

    PE(pos, 2i)   = sin(pos / 10000^(2i/d))
    PE(pos, 2i+1) = cos(pos / 10000^(2i/d))

    Properties:
    - Each position has unique encoding
    - Relative positions can be computed via linear transformation
    - Generalizes to longer sequences than seen during training
```

```
    """
    def __init__(self, d_model, max_len=100, dropout=0.1):
        super().__init__()
        self.dropout = nn.Dropout(p=dropout)

        pe = torch.zeros(max_len, d_model)
        position = torch.arange(0, max_len).unsqueeze(1)
        div_term = torch.exp(
            torch.arange(0, d_model, 2) * (-math.log(10000.0) / d_model)
        )

        pe[:, 0::2] = torch.sin(position * div_term)  # Even indices
        pe[:, 1::2] = torch.cos(position * div_term)  # Odd indices

        self.register_buffer('pe', pe.unsqueeze(0))  # [1, max_len, d_model]

    def forward(self, x):
        # x: [B, L, D]
        x = x + self.pe[:, :x.size(1), :]
        return self.dropout(x)
```

Dimension Flow:

```
Input:  x [B, 20, 152] (text + style embeddings)
     ↓
PE:     [1, 20, 152] (precomputed, added element-wise)
     ↓
Output: [B, 20, 152] (position-aware embeddings)
```

Visualization of PE patterns:

```
Position 0:  [sin(0), cos(0), sin(0/k), cos(0/k), ...]
Position 1:  [sin(1), cos(1), sin(1/k), cos(1/k), ...]
Position 2:  [sin(2), cos(2), sin(2/k), cos(2/k), ...]
...
(Each position has unique "fingerprint")
```

Why for Handwriting?

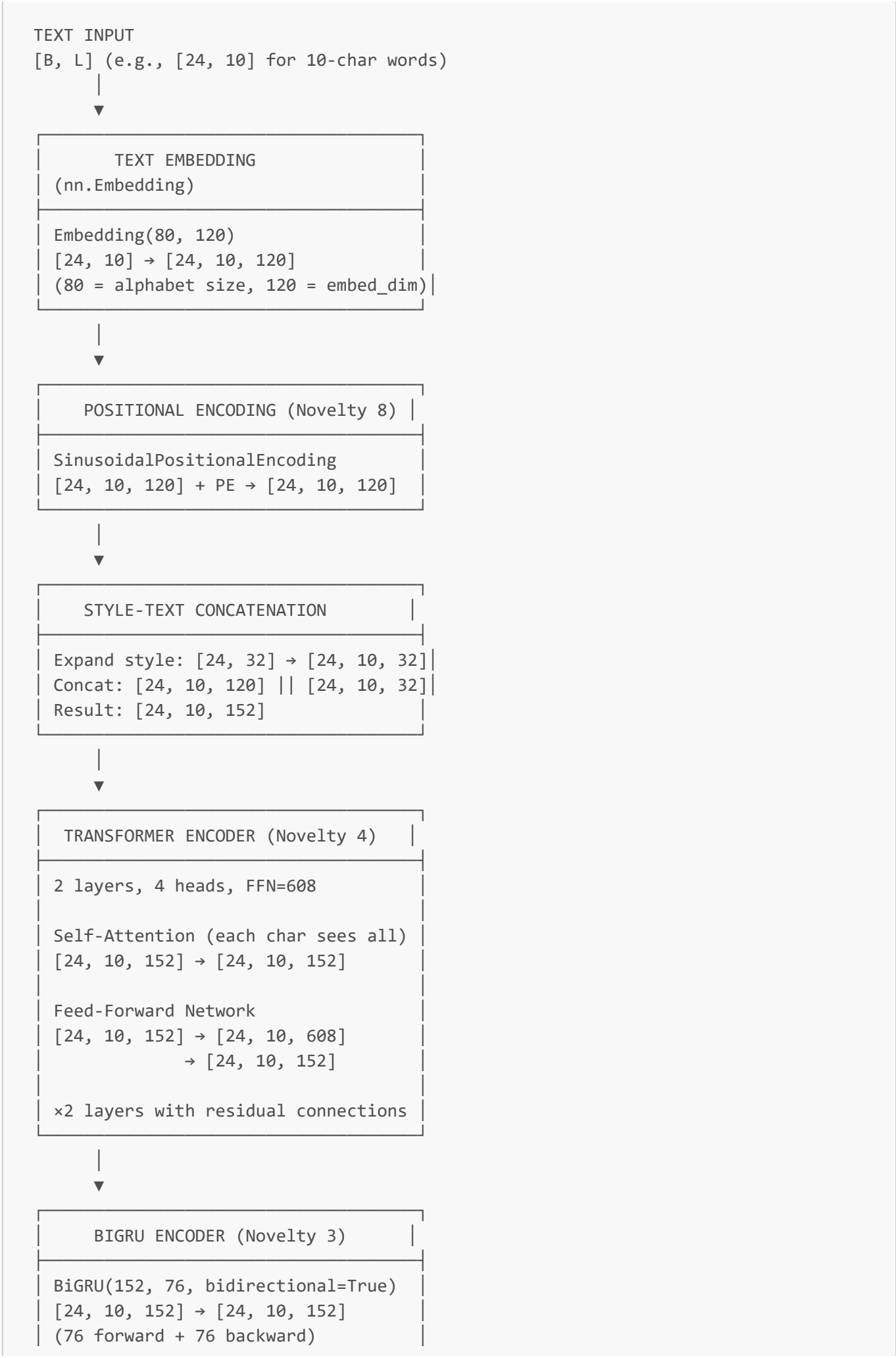
- Ensures consistent left-to-right flow
- Helps with proper character spacing
- Critical for Transformer self-attention to understand sequence order
- Enables generation of variable-length words

🔄 Complete Forward Pass (Step-by-Step)

Phase 1: Style Encoding

```
STYLE IMAGE INPUT
[B, 1, 64, W]  (e.g., [24, 1, 64, 320])
      │
      ▼
┌─────────────────────────────────────────┐
│         STYLE BACKBONE                  │
│  (networks/module.py → StyleBackbone)   │
├─────────────────────────────────────────┤
│ ConstantPad2d + Conv 5×5                │
│ [24, 1, 64, 320] → [24, 16, 32, 160]    │
│                                         │
│ ResBlock ×2 + MaxPool                   │
│ [24, 16, 32, 160] → [24, 64, 16, 80]    │
│                                         │
│ ResBlock ×2 + MaxPool                   │
│ [24, 64, 16, 80] → [24, 128, 8, 40]     │
│                                         │
│ ResBlock ×2 + MaxPool                   │
│ [24, 128, 8, 40] → [24, 256, 4, 20]     │
│                                         │
│ Conv 3×3 (final)                        │
│ [24, 256, 4, 20] → [24, 256, 1, 20]     │
│                                         │
│ Squeeze + Transpose                     │
│ [24, 256, 20] (sequence format)         │
└─────────────────────────────────────────┘
      │
      ▼
┌─────────────────────────────────────────┐
│         STYLE ENCODER                   │
│  (networks/module.py → StyleEncoder)    │
├─────────────────────────────────────────┤
│ Global Average Pool over sequence       │
│ [24, 256, 20] → [24, 256]               │
│                                         │
│ Linear 256→256 + LeakyReLU ×2           │
│ [24, 256] → [24, 256]                   │
│                                         │
│ VAE heads:                              │
│ μ = Linear(256, 32)  → [24, 32]         │
│ logσ² = Linear(256, 32) → [24, 32]      │
│                                         │
│ Reparameterize:                         │
│ z = μ + σ·ε  → [24, 32]                 │
└─────────────────────────────────────────┘
      │
      ▼
STYLE VECTOR z: [24, 32]
```

Phase 2: Text Processing


TEXT INPUT
[B, L] (e.g., [24, 10] for 10-char words)
TEXT EMBEDDING
(nn.Embedding)
Embedding(80, 120)
[24, 10] → [24, 10, 120]
(80 = alphabet size, 120 = embed_dim)
POSITIONAL ENCODING (Novelty 8)
SinusoidalPositionalEncoding
[24, 10, 120] + PE → [24, 10, 120]
STYLE-TEXT CONCATENATION
Expand style: [24, 32] → [24, 10, 32]
Concat: [24, 10, 120] || [24, 10, 32]
Result: [24, 10, 152]
TRANSFORMER ENCODER (Novelty 4)
2 layers, 4 heads, FFN=608
Self-Attention (each char sees all)
[24, 10, 152] → [24, 10, 152]
Feed-Forward Network
[24, 10, 152] → [24, 10, 608]
→ [24, 10, 152]
×2 layers with residual connections
BIGRU ENCODER (Novelty 3)
BiGRU(152, 76, bidirectional=True)
[24, 10, 152] → [24, 10, 152]
(76 forward + 76 backward)

```
└──────────────────────────────────────────┘
      │
      ▼
┌──────────────────────────────────────────┐
│ CROSS-ATTENTION FUSION (Novelty 2)       │
├──────────────────────────────────────────┤
│ Q = content (text), K/V = style          │
│                                          │
│ Q: [24, 10, 152]                         │
│ K: [24, 1, 32] (style expanded)          │
│ V: [24, 1, 32]                           │
│                                          │
│ Attention + FFN                          │
│ [24, 10, 152] → [24, 10, 152]            │
└──────────────────────────────────────────┘
      │
      ▼
FUSED CONTENT+STYLE: [24, 10, 152]
```

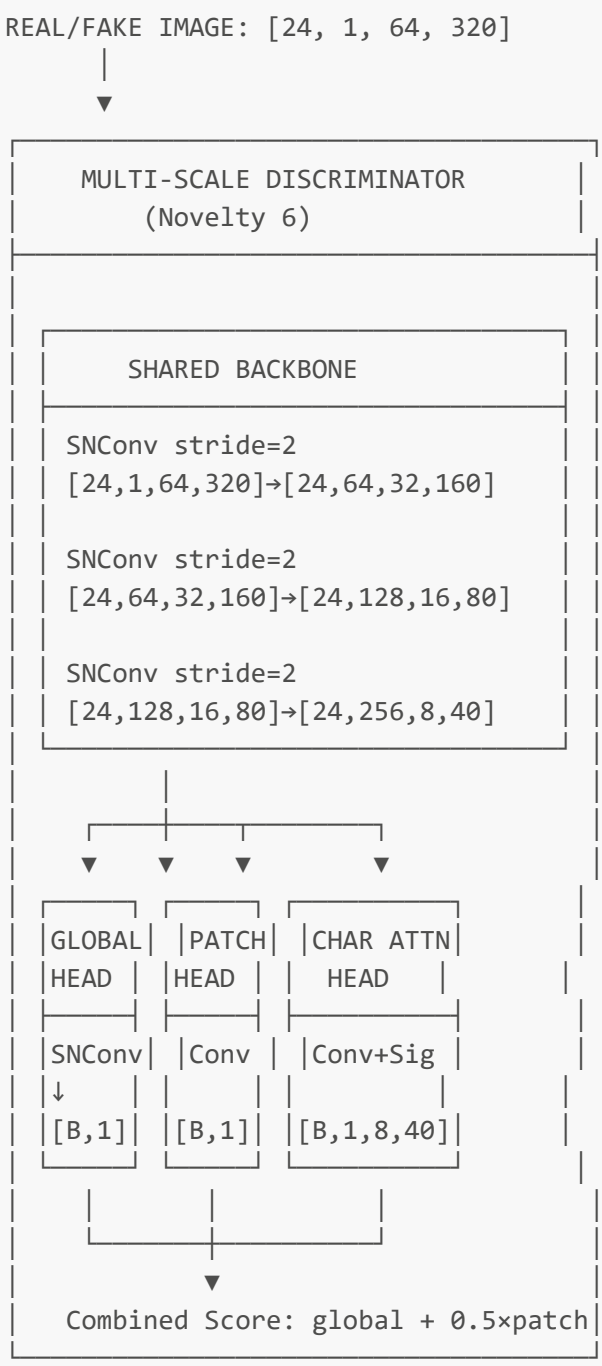
Phase 3: Image Generation

```
FUSED FEATURES: [24, 10, 152]
      │
      ▼
┌──────────────────────────────────────────┐
│       LINEAR PROJECTION                  │
├──────────────────────────────────────────┤
│ Linear(152, 512×4×4)                     │
│ [24, 10, 152] → [24, 10, 8192]           │
└──────────────────────────────────────────┘
      │
      ▼
┌──────────────────────────────────────────┐
│         RESHAPE TO 4D                    │
├──────────────────────────────────────────┤
│ View + Permute                           │
│ [24, 10, 8192]                           │
│  → [24, 40, 4, 512] (L×W, H, C)          │
│  → [24, 512, 4, 40]  (B, C, H, W)        │
│                                          │
│ Initial: 4 pixels height, 4×L width      │
└──────────────────────────────────────────┘
      │
      ▼
┌──────────────────────────────────────────┐
│   ADAIN GBLOCK 1 (Novelty 1)             │
├──────────────────────────────────────────┤
│ Upsample: ×(2,1) → [24, 512, 8, 40]      │
│ Conv + AdaIN + Conv + AdaIN              │
│ [24, 512, 8, 40] → [24, 256, 8, 40]      │
│                                          │
```

```
│ Style injection via:                  │
│ - Instance norm per sample            │
│ - Style-specific scale/shift          │
└──────────────────────────────────────┘
      │
      ▼
┌──────────────────────────────────────┐
│   ADAIN GBLOCK 2 + ATTENTION         │
├──────────────────────────────────────┤
│ Upsample: ×(2,2) → [24,256,16,80]    │
│ Conv + AdaIN + Conv + AdaIN          │
│ [24, 256, 16, 80] → [24, 128, 16, 80]│
│                                      │
│ Multi-Head Self-Attention            │
│ [24, 128, 16, 80] → [24, 128, 16, 80]│
│ (Spatial attention for coherence)    │
└──────────────────────────────────────┘
      │
      ▼
┌──────────────────────────────────────┐
│   ADAIN GBLOCK 3                     │
├──────────────────────────────────────┤
│ Upsample: ×(2,2) → [24,128,32,160]   │
│ Conv + AdaIN + Conv + AdaIN          │
│ [24, 128, 32, 160] → [24, 64, 32, 160]│
│                                      │
│ Skip connection from style backbone  │
│ (Novelty 7: Multi-scale retention)   │
└──────────────────────────────────────┘
      │
      ▼
┌──────────────────────────────────────┐
│   ADAIN GBLOCK 4 + ATTENTION         │
├──────────────────────────────────────┤
│ Upsample: ×(2,2) → [24,64,64,320]    │
│ Conv + AdaIN + Conv + AdaIN          │
│ [24, 64, 64, 320] → [24, 64, 64, 320]│
│                                      │
│ Multi-Head Self-Attention            │
│ (Final coherence check)              │
└──────────────────────────────────────┘
      │
      ▼
┌──────────────────────────────────────┐
│        OUTPUT LAYER                  │
├──────────────────────────────────────┤
│ InstanceNorm2d                       │
│ ReLU                                 │
│ Conv2d(64, 1, 3×3)                   │
│ Tanh                                 │
│                                      │
│ [24, 64, 64, 320] → [24, 1, 64, 320]│
└──────────────────────────────────────┘
      │
```

```
      ▼
GENERATED IMAGE: [24, 1, 64, 320]
(Range: [-1, 1], normalized grayscale)
```

Phase 4: Discrimination



Loss Functions

Generator Losses

Loss	Weight	Purpose	Formula

Loss	Weight	Purpose	Formula
GAN Loss	1.0	Fool discriminator	$-\mathbb{E}[D(G(z,y))]$
Reconstruction L1	1.0	Match real images	$\|G(z,y) - x_{real}\|_1$
KL Divergence	0.0005	VAE regularization	$D_{KL}(q(z\|x) \|\| p(z))$
Perceptual	2.0	Feature matching	$\sum_l \|VGG_l(fake) - VGG_l(real)\|_1$
Contextual (CX)	1.5	Style similarity	$-\log(CX(fake, real))$
Gram (Style)	2.5	Texture matching	$\|Gram(fake) - Gram(real)\|_F$
OCR	1.0	Text correctness	$CTC(OCR(fake), text)$
Writer ID	1.0	Writer consistency	$CE(WID(fake), writer_{id})$
Contrastive	0.1	Style disentanglement (Novelty 5)	InfoNCE

Discriminator Losses

Loss	Weight	Purpose
Real/Fake	1.0	Standard adversarial
R1 Regularization	10.0	Gradient penalty
Consistency	0.1	Multi-scale agreement (Novelty 6)

Hyperparameters

```yaml
# Model Architecture
style_dim: 32           # Style vector dimension
embed_dim: 120          # Text embedding dimension
G_ch: 64                # Generator base channels
D_ch: 64                # Discriminator base channels
resolution: 64          # Image height
char_width: 32          # Pixels per character

# Transformer (Novelty 4)
transformer_layers: 2
transformer_heads: 4
transformer_ff_dim: 608  # 4 × (embed_dim + style_dim)

# BiGRU (Novelty 3)
bigru_layers: 1
bigru_hidden: 152       # Same as input

# Cross-Attention (Novelty 2)
cross_attn_heads: 4
cross_attn_dropout: 0.1

# Training
```

```
batch_size: 24
learning_rate: 1.5e-4
adam_beta1: 0.5
adam_beta2: 0.999
num_critic_train: 5     # D updates per G update
epochs: 70

# Contrastive (Novelty 5)
contrastive_temperature: 0.07
```

📊 Output Examples

Dimension Summary

Stage	Tensor Shape	Description
Input Image	`[B, 1, 64, W]`	Grayscale, height=64, variable width
Style Vector	`[B, 32]`	Compressed style representation
Text Indices	`[B, L]`	Character indices (max 20)
Text Embedding	`[B, L, 120]`	Dense embeddings
Fused Features	`[B, L, 152]`	Style + text combined
Initial Feature Map	`[B, 512, 4, 4L]`	Before upsampling
Block 1 Output	`[B, 256, 8, 4L]`	After first GBlock
Block 2 Output	`[B, 128, 16, 8L]`	After second GBlock
Block 3 Output	`[B, 64, 32, 16L]`	After third GBlock
Block 4 Output	`[B, 64, 64, 32L]`	After fourth GBlock
Generated Image	`[B, 1, 64, 32L]`	Final output

Width Calculation

For a word with L characters: $$W_{out} = L \times char_width = L \times 32$$

Example: "hello" (5 chars) → 160 pixels wide

🔍 Why These Novelties for Handwriting?

Novelty	Handwriting Problem	Solution
AdaIN + ModConv	Writers have unique stroke weights, slants	Per-sample style modulation
Cross-Attention	Different chars need different style emphasis	Selective style application
BiGRU	Ligatures, consistent flow	Sequential dependencies

Novelty	Handwriting Problem	Solution
Transformer	Global word structure	All-to-all character relations
Contrastive	Mixing writer identity with content	Disentangled style space
Multi-Scale D	Both global structure and local texture matter	Specialized discrimination
Skip-Connections	Fine strokes lost in deep layers	Multi-scale style injection
Positional Encoding	Character order and spacing	Position-aware generation

References

- **Original HiGAN**: Handwriting Imitation GAN (CVPR 2021)
- **StyleGAN2**: Analyzing and Improving the Image Quality of StyleGAN
- **BigGAN**: Large Scale GAN Training for High Fidelity Natural Image Synthesis
- **Transformer**: Attention Is All You Need
- **InfoNCE**: Representation Learning with Contrastive Predictive Coding

---

Architecture documentation generated for the HiGAN+ handwriting style transfer system. Last updated: November 2025